APF-CPP: An Artificial Potential Field Based Multi-Robot Online Coverage Path Planning Approach

Zikai Wang, Xiaoqi Zhao, Jiekai Zhang, Nachuan Yang, *Member, IEEE*, Pengyu Wang, *Graduate Student Member, IEEE*, Jiawei Tang, *Graduate Student Member, IEEE*, Jiuzhou Zhang, and Ling Shi, *Fellow, IEEE*

***Abstract*—Multi-robot coverage planning has gained significant attention in recent years. In this letter, we introduce a novel approach called APF-CPP (Artificial Potential Field Based Multi-Robot Online Coverage Path Planning) to enhance the collaboration of multi-robot systems to accomplish coverage tasks in unknown dynamic environments. Our approach presents a unique coverage policy that leverages the concept of artificial potential field (APF). In contrast to the conventional APF-based path planning methods that directly generate paths based on the field gradient, we utilize the APF to derive coverage policies for individual robots within a multi-robot system to achieve efficient task allocation and maintain regular coverage patterns. We have developed a policy update mechanism that allows the system to adapt its task allocation policy based on real-time conditions while minimizing the impact caused by policy changes. To better handle dead-end conditions, we use the APF concept to allocate tasks better during the dead-end recovery process. We also show that our algorithm has a low computational complexity and guarantees complete coverage in a finite time. We conduct extensive comparisons with other state-of-the-art (SOTA) approaches and validate our method through simulations and real-world experiments. The experimental results demonstrate the advantages of our proposed method over existing approaches and confirm the effectiveness and robustness of real-world implementation.**

***Index Terms*—Autonomous agents, multi-robot systems, path planning for multiple mobile robots or agents, planning, planning under uncertainty, scheduling and coordination.**

Manuscript received 19 March 2024; accepted 12 July 2024. Date of publication 23 July 2024; date of current version 18 September 2024. This article was recommended for publication by Associate Editor Jun-Ho Lee and Editor Chao-Bo Yan upon evaluation of the reviewers' comments. *(Corresponding author: Nachuan Yang.)*

Zikai Wang, Xiaoqi Zhao, Nachuan Yang, Jiawei Tang, Jiuzhou Zhang, and Ling Shi are with the Department of Electronic and Computer Engineering, Hong Kong University of Science and Technology, Clear Water Bay, Hong Kong (e-mail: zwanggz@connect.ust.hk; xzhaobb@connect.ust.hk; nc.yang@connect.ust.hk; jtangas@connect.ust.hk; jzhanghz@connect.ust.hk; eesling@ust.hk).

Jiekai Zhang is with the Hong Kong Applied Science and Technology Research Institute Company Limited, Shatin, Hong Kong (e-mail: jiekaizhang@astri.org).

Pengyu Wang is with the Department of Electronic and Computer Engineering, Hong Kong University of Science and Technology, Clear Water Bay, Hong Kong, and also with Shenzhen Key Laboratory of Robotics Perception and Intelligence, Department of Electronic and Electrical Engineering, Southern University of Science and Technology, Shenzhen 518055, China (e-mail: pwangat@connect.ust.hk).

This letter has supplementary downloadable material available at https://doi.org/10.1109/LRA.2024.3432351, provided by the authors.

Digital Object Identifier 10.1109/LRA.2024.3432351

I. Introduction

A. Background

COVERAGE path planning using multi-robot systems has garnered attention in recent years for their ability to handle large-scale tasks, improve robustness, and enhance coverage efficiency compared to a single robot [1]. Multi-robot coverage path planning has numerous applications, such as floor cleaning [2], environmental monitoring [3], smart agriculture [4], disaster response [5], and search and rescue [6]. The multi-robot online coverage path planning problem involves efficiently coordinating multiple robots to achieve complete coverage of a target area while considering minimal coverage cost and adaptability to environmental changes or robot malfunctions [7]. While there have been numerous advancements and fascinating applications in this field, there is still an ongoing and open research topic on further enhancing the performance.

B. Related Works

One way to classify coverage path planning algorithms is based on the type of information utilized for planning [8]. Offline algorithms assume the environmental information for path planning is completely known in the priority of the coverage process, and the environment is assumed to be stationary during the process. On the other hand, online algorithms do not require complete knowledge of the environment, and the planner makes decisions based on real-time detection of the environment.

Offline coverage methods are generally easier to implement and more straightforward. They can finish the overall planning before the coverage process and follow the pre-planned path to finish the coverage task. In [9], the authors transformed the single-robot spanning tree coverage (STC) Algorithm into a multi-robot version. Tang et al. [10] proposed the MSTC* algorithm, which incorporates strict physical constraints and can find a set of well-balanced workload distributions. In [11], the authors enhanced MSTC* to minimize the number of turns of the coverage paths. In [12], minimizing the number of turns was also considered as the main objective. A method was developed in [13] to obtain equal coverage routes for robots while guaranteeing obstacle avoidance. Tushar Kusnur and Maxim Likhachev [14] presented a decomposition-free approach to complete coverage

2377-3766 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: International Institute of Information Technology Bangalore. Downloaded on April 14,2026 at 17:42:39 UTC from IEEE Xplore. Restrictions apply.

path planning. Song et al. [15] proposed a method to carry out multi-UAV exploration tasks under energy constraints. However, offline methods are not robust because each member's task allocation and coverage path are pre-determined before the process starts. The original path can become inefficient or ineffective when encountering unexpected environmental changes.

Online coverage presents a more significant challenge than offline coverage. Each robot must plan and execute its coverage task simultaneously, even when complete knowledge of the target coverage environment is unavailable. Still, this kind of method is more robust. Galceran et al. [16] presented a real-time approach to planning 3D coverage paths. In [17], the authors used reinforcement learning to learn a proper policy to adapt to real-time environmental information. Liu et al. [18] proposed a priority template and a novel global backtracking mechanism to improve multi-robot coverage efficiency. In [19], the authors proposed a receding horizon centralized online planner to handle the online multi-robot coverage problem. Hu et al. [20] defined a utility function for selecting frontier points and used Voronoi partitions to allocate coverage tasks among robots. In [21], the authors expanded the single-robot online coverage path planning method PPCPP [22] into a decentralized method for online multi-robot coverage path planning problems. Song and Gupta [23] designed a cooperative game for optimal task reallocation of the multi-robot system during the coverage process in unknown environments. In [24], the same authors expanded single robot ϵ^* [25] into a multi-robot version named CARE. This method provides resilience to the robot team against failures of individual robots and also improves overall efficiency via event-driven re-planning.

The shortcomings of existing methods are as follows: First, the existing methods often require a careful arrangement of the robots' initial setup before the coverage task. Typically, this involves evenly dispersing the robots across the target region or manually allocating them to specific areas. However, such requirements can be impractical or even unrealistic in many real-world implementations. Second, existing algorithms rely heavily on manual adjustment of various factors based on engineers' experience and the setup of target coverage regions. Failure to appropriately configure these factors can significantly compromise coverage efficiency. This manual adjustment requirement poses a challenge for users without extensive experience in the field. Third, current methods have not paid enough attention to the different working capabilities (like movement speed) among the multi-robot system. Simply assuming robots have the same capability can lead to unfair task allocation and low coverage efficiency.

C. Main Contributions

In this letter, we propose a novel multi-robot online coverage path planning approach called APF-CPP to address the above-mentioned challenges. The contributions of the paper are summarized as follows:

- We introduce an online coverage path planner for the multi-robot system with diverse working speeds, leveraging the APF concept for efficient task allocation without relying on human setup.
- We propose a new concept of coverage policy and a novel coverage policy update mechanism that can achieve reasonable collaboration among the multi-robot system and adapt to real-time working conditions.
- We have devised a novel dead-end recovery mechanism to address situations where robots encounter dead-ends effectively.

II. The APF-CPP Approach

A. Problem Formulation

Consider a group of N mobile robots R_i, $i \in \{1, 2, \ldots, N\}$ with different operating speeds and square (of sides d) coverage tool footprint deployed in the target coverage area. In this letter, we assume that the working area is either obstructed by obstacles or defined in advance by humans. The area map can be acquired beforehand or built progressively as the coverage process unfolds. The stability of the environment is not guaranteed. For convenience, we divide the target coverage area into cells of size $d \times d$. The cells are represented as q, q is the coordinate of the cell's center, and the set of cells is represented as Q. The cells in the target coverage terrain are divided into three types:

1) Covered cell q_c: if the center of coverage tool footprint overlaps with a cell's center, the cell is defined as covered, Q_c is defined as the set of covered cells.
2) Occupied cell q_o: if a cell is occupied by obstacles or is set as a no-entry zone that the robot cannot access. It is defined as occupied, and the set of occupied cells is defined as Q_o.
3) Uncovered cell q_u: if the center of an unoccupied cell has not been overlapped with the coverage footprint center of any robot, it is defined as uncovered. Q_u is defined as the set of uncovered cells. If the status of the cells in the target terrain is unknown, we set them as uncovered cells.

Therefore, the overall set of cells that need to be covered is $Q_l = Q \backslash Q_o$, which can be changed during the coverage process for unexpected environmental changes. Cell q_{R_i} is defined as R_i's current cell if the coverage footprint center of R_i is located in that cell.

Definition 1 (Neighbor cells): Two cells q_i and q_j are considered neighbor cells if:

$$\|q_i - q_j\| \leq d_n, \tag{1}$$

where d_n is typically set as d or $\sqrt{2}d$ depending on the robot's mobility and environment setup, indicating the 4 or 8 neighboring cells. The set of neighbor cells for cell q_i is defined as $Q^n_{q_i}$.

Definition 2 (Dead-end condition): The robot R_i is in a dead-end condition if:

$$q \neq q_u, \forall q \in Q^n_{q_{R_i}}. \tag{2}$$

This means all R_i's neighboring cells are already occupied or covered under the dead-end condition. q_{R_i} is defined as a dead-end.

Authorized licensed use limited to: International Institute of Information Technology Bangalore. Downloaded on April 14,2026 at 17:42:39 UTC from IEEE Xplore. Restrictions apply.

Definition 3 (Boundary cell): A cell q_i is defined as a boundary cell if:

$$q_i = q_u \wedge \exists q \in Q^n_{q_i} \text{ s.t. } q = q_c. \tag{3}$$

This means if a cell possesses covered neighbor cells, it is a boundary cell, and the set of boundary cells is called the boundary list, defined as L.

The problem of multi-robot online coverage path planning can be defined as follows: given a workspace Q and a set of N robots R, a real-time planner that allows each robot to cover all the accessible cells $Q_l \subseteq Q$, i.e., $Q_c = Q_l$. The planner should be computationally tractable and adapt to environmental changes while minimizing coverage costs such as coverage time and total path length.

We assume the robot in the multi-robot system can get its global position in the working area. The robot can detect obstacles in its detection range and communicate with the central server.

B. Overview of APF-CPP

In brief, the APF-CPP approach is used to determine the movement target cell q^{tar} for robots in the system. R_i's target cell q_i^{tar} can only be chosen from $Q^n_{q_{R_i}}$. If R_i is not in the dead-end condition, it will select an uncovered neighbor cell based on its coverage policy as its q_i^{tar}. A target selection mechanism will be introduced in the Section II-C. If the robot falls into a dead-end condition, it will use a dead-end recovery mechanism to determine its q_i^{tar} to jump out from this condition and continue the coverage process. This mechanism is illustrated in Section II-D. When the robot reaches its target, a new target will be selected. As a result, the planner will update the robots' targets iteratively based on the information received from the multi-robot system during the coverage process. This makes the system more adaptive to environmental changes. The pseudo-code in Algorithm 1 shows the details of the APF-CPP in a centralized entity.

In the centralized approach, the central server manages and distributes information to each robot, allowing for coordinated decision-making and ensuring that all robots work together toward the common goal of coverage. The central server maintains L, Q_u, Q_c, Q_o, a set of all robots' current locations P, and a set of all robots' target cells Q^{tar}. The only requirement of the algorithm is the velocity set V, which contains all the robots' operational speed v. At the beginning of the coverage process, the position of each robot p_i is obtained from the positioning device, and each robot's coverage policy is determined. The global boundary list L is initialized as empty, and each robot's target cell q_i^{tar} is set to its position p_i. Q_u is initialized as all accessible cells Q_l in the target area. The coverage process continues until Q_u becomes empty.

When R_i reaches q_i^{tar}, it will send an arrival message and its observation to the central server. The central server then sets q_{R_i} as q_c and updates Q_c, Q_u and Q_o. Next, the server evaluates the status of $Q^n_{q_{R_i}}$ and updates L (Algorithm 1: line 18 to 24). Fig. 1 shows the process of boundary list maintenance. Subsequently, if R_i is in a dead-end condition, the planner updates R_i's coverage policy $policy(i)$ and uses the dead-end recovery mechanism to determine q_i^{tar}. Otherwise, it selects q_i^{tar} based on R_i's current coverage policy (Algorithm 1: line 25 to 30). Then, the central server sends the newly planned q_i^{tar} to R_i.

Algorithm 1: APF-CPP.

1: **Require:** V
2: **Initialize:** $L = \emptyset$, $Q_u = Q_l$, $Q_c = \emptyset$, $Q_o = \emptyset$
3: **for** $i \in \{1, 2, \ldots, N\}$ **do**
4: $q_i^{tar} = p_i$
5: $policy(i) \leftarrow$ PolicyUpdate
6: **end for**
7: **while** $Q_u \neq \emptyset$ **do**
8: ▷ update P, Q_o, Q_c, Q_u
9: $P \leftarrow$ information from $R_i, \forall i \in \{1, 2, \ldots, N\}$
10: **if** R_i reaches q_i^{tar} **then**
11: $Q_c \leftarrow Q_c \cup \{q_{R_i}\}$
12: **for** $q \in$ cells within R_i's observation region **do**
13: **if** $q = q_0$ and $q \notin Q_o$ **then**
14: $Q_o \leftarrow Q_o \cup \{q\}$
15: $Q_u \leftarrow Q_u \backslash \{q\}$
16: **else if** $q \neq q_c$ and $q \neq q_o$ and $q \notin Q_u$ **then**
17: $Q_u \leftarrow Q_u \cup \{q\}$
18: **end if**
19: **end for**
20: ▷ Update the boundary list
21: **for** $q \in Q^n_{R_i}$ **do**
22: **if** $q = q_u$ and $q \notin L$ **then**
23: $L \leftarrow L \cup \{q\}$
24: **else if** $q = q_c$ and $q \in L$ **then**
25: $L \leftarrow L \backslash \{q\}$
26: **end if**
27: **end for**
28: **if** Dead-end condition **then**
29: ▷ Handle the Dead-end condition
30: $policy(i) \leftarrow$ PolicyUpdate
31: $q_i^{tar} \leftarrow$ DeadEndRecover
32: **else**
33: ▷ Choose the target from neighbor cells
34: $q_i^{tar} \leftarrow$ NextTarget($policy(i)$)
35: **end if**
36: update Q^{tar}
37: send q_i^{tar} to R_i
38: **end if**
39: **end while**

C. Coverage Policy

For R_i, if there are uncovered cells in $Q^n_{q_{R_i}}$, the planner will choose the cell with the shortest distance as q_i^{tar}. When there are uncovered neighbor cells with the equal closest distance to the robot, the planner will determine the target cell according to $policy(i)$. The robot's coverage policy is a task allocation mechanism, which guides the robot's overall coverage direction. The robot's coverage policy is determined by the direction of total repulsive force f_i from all the other robots in the system.

Authorized licensed use limited to: International Institute of Information Technology Bangalore. Downloaded on April 14,2026 at 17:42:39 UTC from IEEE Xplore. Restrictions apply.

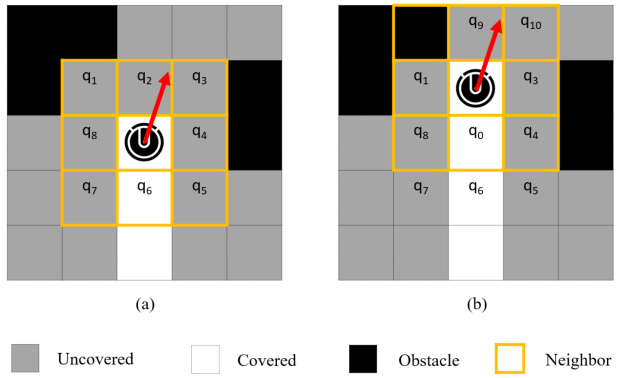


Fig. 1. The image shows how the coverage policy and boundary list maintenance works. In Fig. 1(a), the robot moves from cell q_6 to its current location q_0. The robot's coverage policy is shown in the direction of the red arrow. The planner will select q_2 as the robot's q^{tar}. After arriving at cell q_2, as shown in Fig. 1(b), cells q_9 and q_{10} will be added to the boundary list, and cell q_0 will be removed from the boundary list, then q_9 will be selected as the robot's q^{tar}.

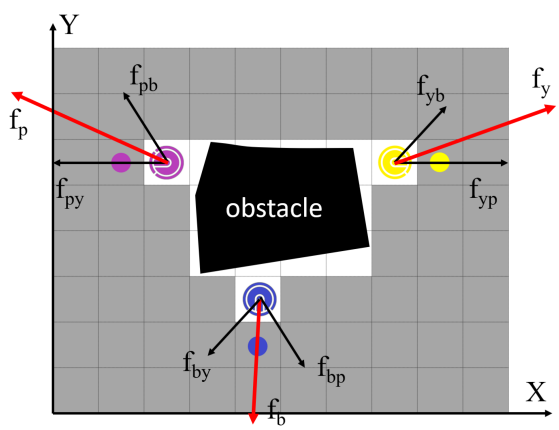


Fig. 2. An illustration of the policy update. Three robots are initially deployed in the area. The black arrows represent the repulsive force of other robots, and the red arrows represent the total force acting on the robot. Then, the coverage policy of the pink, blue, and yellow robot is the direction of f_p, f_y and f_b. The dots are the target points for robots with the same color.

The repulsive force f_{ij} from robot j on robot i is defined as:

$$f_{ij} = \frac{\|v_j\|\,(p_j - p_i)}{\|p_j - p_i\|^2}, \tag{4}$$

while the total force is defined as:

$$f_i = \sum_{j \neq i} f_{ij}, \tag{5}$$

where p_i and p_j represent the position of robots i and j. v_j denotes the operating speed of robot j. As shown in Fig. 2, for the pink robot, f_{pb} is the repulsive force from the blue robot, f_{py} is the repulsive force from the yellow robot, f_p is the total repulsive force on the pink robot.

When there are uncovered neighbor cells with equal distance to q_{R_i}, the robot will select the cell q with the smallest angle θ between $q - q_{R_i}$ and f_i as q^{tar}, in which θ is represented as:

$$\theta = \arccos\left(\frac{(q - q_{R_i}) f_i}{\|(q - q_{R_i})\|\,\|f_i\|}\right). \tag{6}$$

Algorithm 2: Dead-end Recovery.

Require: P, V, L
Initialize: $l_i = \emptyset$
▷ Calculate l_i
for q in L **do**
  Calculate f^a_{iq} according to (7) $\forall i \in \{1, 2, \ldots, N\}$
  if $\|f^a_{iq}\| > \|f^a_{jq}\|, \forall j \in \{1, 2, \ldots, N\} \backslash i$ **then**
    $l_i \leftarrow l_i \cup \{q\}$
  end if
end for
▷ Determine the goal cell and calculate the target cell
if $l_i = \emptyset$ **then**
  $G_i \leftarrow$ the cell with the largest f^a_i in L
  $q^{tar}_i \leftarrow$ Point-to-Point planner
else
  $G_i \leftarrow$ the cell with the largest f^a_i in l_i
  $q^{tar}_i \leftarrow$ Point-to-Point planner
end if

The target cell selection according to coverage policy is represented in Fig. 2.

However, frequent changes in the coverage policy will lead to compromised coverage performance. Consequently, we propose a coverage update mechanism that restricts the modification of the coverage policy to the initiation of coverage tasks or when the robot encounters a dead-end.

By implementing the coverage policy based on repulsive forces, the overall movement direction of the robots can be determined, leading to their dispersion and fostering improved cooperation in the multi-robot system. The update mechanism of the coverage policy only needs the operational speeds of robots and does not rely on artificial tuning.

D. Dead-End Recovery

During the coverage process, the robot may end up in a cell where all neighboring cells are already occupied or covered, known as the dead-end condition. The dead-end recovery mechanism is developed to tackle this scenario, and the pseudocode in Algorithm 2 shows the details. When R_i reaches a dead-end, the attractive force is generated to determine the backtrack goal G_i from L. The attractive force from cell q on robot R_i is defined as:

$$f^a_{iq} = \frac{\|v_i\|(q - q_{R_i})}{\|q - q_{R_i}\|^2}, \tag{7}$$

where q is a cell in L, v_i is the operation speed of R_i. The attractive force from cell q on R_i is proportional to the magnitude of R_i's operational speed and is inversely proportional to the distance between q and q_{R_i}. According to f^a_{iq}, R_i's backtrack list l_i is proposed, which is defined as:

$$l_i = \left\{q \in L \mid \|f^a_{iq}\| > \|f^a_{jq}\|, \forall j \neq i\right\}. \tag{8}$$

The planner can only choose G_i from the set of cells that generate a larger attractive force than other robots in the system

Authorized licensed use limited to: International Institute of Information Technology Bangalore. Downloaded on April 14,2026 at 17:42:39 UTC from IEEE Xplore. Restrictions apply.

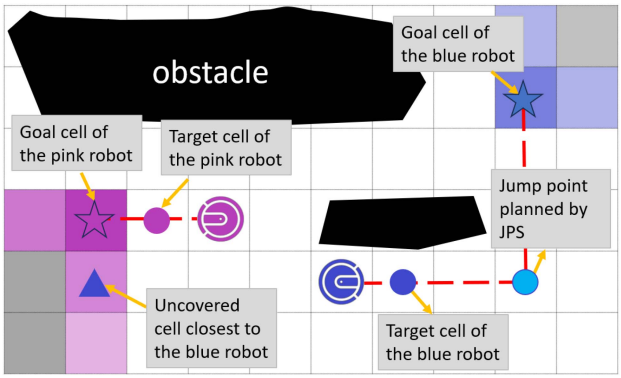


Fig. 3. The image depicts the division of L. The pink and blue robots are in a dead-end condition as their neighbors are all covered (white) cells, while the blue and pink cells represent entries in the backtrack list l_{pink} and l_{blue}. The darkness of the color represents the magnitude of the attractive force. Consequently, the pink robot is allocated the cell with the darkest color as G_{pink}. It uses the JPS planner to generate the jump points to G_{pink}, and the neighbor cell with the lowest distance to the next jump point is selected as the q_{pink}^{tar}. The blue robot follows the same procedure.

(Algorithm 2: line 2 to 7). When $l_i = \emptyset$, the robot will choose G_i in L that generates the largest attractive force on it.

After determining the backtrack goal, the server will use the point-to-point path planner to generate q_i^{tar} (Algorithm 2: line 8 to 14). Here, we use the JPS method [26] to generate the jump points to G_i, and q_i^{tar} is selected as the neighbor cell with the lowest distance to the next jump point.

During the process of R_i reaching its backtrack goal, G_i is updated at each time step since the coverage status and environment can change. The coverage task resumes once the robot is out of the dead-end condition. The process of dead-end recovery is shown in Fig. 3. Both robots in the figure have fallen into a dead-end. To recover from this condition, the blue robot is allocated a cell further away from itself rather than some closer pink cells as the backtrack goal. This leads to a faster completion time covering the target area than allocating the robots to the nearest cell as the backtrack goal.

E. Uncertainty-Handling Mechanism

The robots, acting as executors of a centralized algorithm, must deal with uncertainties from the environment during the coverage task. These uncertainties include issues like communication failures between robots and the central server and the sudden appearance of dynamic obstacles. To address these unexpected situations, an uncertainty-handling mechanism for the executors is designed.

Upon reaching q_i^{tar}, robot R_i halts and sends a planning request to the server. In the event of communication uncertainties, the robot persistently sends messages at a predefined frequency until it receives the newly planned target from the server. After receiving q_i^{tar}, the R_i uses its local planner to navigate towards it. If unexpected obstacles are in the way of the local path and are within a predefined safety range, the robot halts immediately to avoid collisions until the obstacles clear. If the obstacles persist beyond a predefined time, the robot requests the server to replan and provide a new target. This approach improves the multi-robot system's robustness in real-world implementations.

III. Algorithm Analysis

A. Coverage Completeness in Finite Time

We assume the target coverage region is connected; there are no setup changes in the undetectable area, and none of the robots will break down. The APF-CPP algorithm provides a guarantee of achieving complete coverage in a finite time. This is achieved through the following mechanisms:

- Stopping Criterion: The algorithm continues until Q_u becomes empty, which promises completeness.
- Cell Status Update: At each step, R_i reaches q_i^{tar}, Q_c will be updated, and after planning new q_i^{tar}, Q_{tar} will be updated to avoid duplicated coverage and target collision.
- Dead-End Recovery: When falling into a dead-end condition, this mechanism will find a backtrack goal for the robot to jump out from this condition. This prevents the robots from getting stuck in revisiting already covered cells.

By incorporating these mechanisms, the APF-CPP algorithm ensures that if uncovered cells are in the target coverage region, the algorithm will not stop, guaranteeing completeness. Each time the robot plans, it will head to an uncovered cell until the coverage is completed. There is no condition that the robots fall into a dead loop in which the robot repeatedly covers the already covered cells, guaranteeing that the algorithm can finish the task in a finite time.

B. Computational Complexity

For the update of L (Algorithm 1: line 18 to 24) and target choosing from the neighbor cells (Algorithm 1: line 29), the time complexity only depends on the number of neighbor cells, so the time complexity is $O(n)$, where n is the number of neighbor cells. When the robot falls into a dead-end condition, the time complexity depends on the point-to-point planner (Algorithm 2: line 8 to 14). Note that the dead-end situation happens only a small number of times during the entire coverage task. The time complexity of the 'can and update' procedure (Algorithm 1: line 10 to 16) is needed for all algorithms; therefore, the analysis of these two parts does not consider these procedures.

As a result, the time complexity of one step of planning the proposed algorithm (not including the dead-end recovery or 'scan and update' procedure) is $O(n)$, which means the planning can be finished in a very short time.

IV. Experiments and Discussions

A. Simulation Experiments

In this section, we conduct two case studies to illustrate the performance of APF-CPP and compare it with BNNB [18] and Dec-PPCPP [21]. We aim to demonstrate the differences in total coverage path length, completion time, and average turning ratio (average turning time of each robot divided by the completion time) for these algorithms in unknown environments

Authorized licensed use limited to: International Institute of Information Technology Bangalore. Downloaded on April 14,2026 at 17:42:39 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPUTATION TIME ANALYSIS: APF-CPP UNDER VARYING NUMBER OF ROBOTS

Modes	3 robots	6 robots	9 robots	12 robots	15 robots
peak	116.43 ms	109.37 ms	128.12 ms	140.62 ms	125.00 ms
avg	0.57 ± 0.15 ms	0.75 ± 0.22 ms	1.27 ± 0.25 ms	2.12 ± 0.76 ms	3.08 ± 0.73 ms
avg total	4.50 ± 1.52 s	6.33 ± 2.31 s	12.11 ± 2.73 s	21.92 ± 10.28 s	33.47 ± 10.66 s

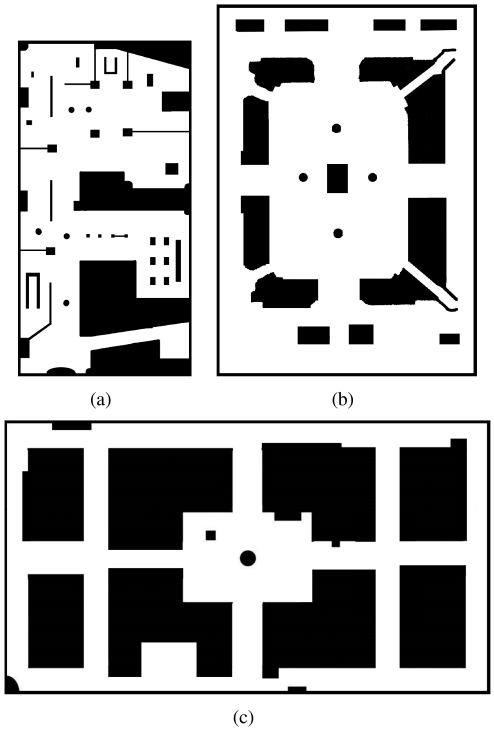


Fig. 4. Three coverage regions with different sizes and setups. The white areas represent the accessible areas, and the black areas represent the occupied areas.

TABLE II
COMPUTATION TIME ANALYSIS: APF-CPP UNDER VARYING TARGET AREA SIZES

Modes	Map 2	Map 3
peak	203.12 ms	265.62 ms
avg	2.94 ± 0.58 ms	6.08 ± 0.76 ms
avg total	56.15 ± 15.20 s	122.71 ± 26.67 s

with different numbers of robots and under varying workspaces. All simulations were carried out using Python environments on an Intel i7-10700F CPU.

Three target coverage environments were represented by maps depicted in Fig. 4: a shopping mall (75 m × 40 m) shown as Fig. 4(a), a square (90 m × 60 m) shown as Fig. 4(b), and a building corridor (120 m × 70 m) shown as Fig. 4(c). The maps were divided into small square cells measuring 0.5 m x 0.5 m, matching the robots' coverage footprint. The number of elements for Q_l in each map was 8530 for the shopping mall, 16776 for the square, and 16701 for the building corridor.

1) Case Study 1: Coverage With Different Numbers of Robots: The experiment region is map 1, shown in Fig. 4(a). Each set of experiments is divided into five groups with different numbers of participating robots. The groups were 3, 6, 9, 12, and 15 robots. In each group, one-third of the robots had an operational speed of 0.5 m/s, another third had a speed of 0.25 m/s, and the remaining third had a speed of 0.15 m/s. All the robots have the same rotation speed of 0.25 rd/s, and the detection range of each robot is set at 5 m. The robots' initial positions were squeezed around the left corner of the map. For each group, 30 tests were carried out. We assume the system does not know these coverage regions and needs to construct the region map during coverage. The status of cells in the unknown area is set as uncovered. We started the experiment when all the robots arrived at their initial cell.

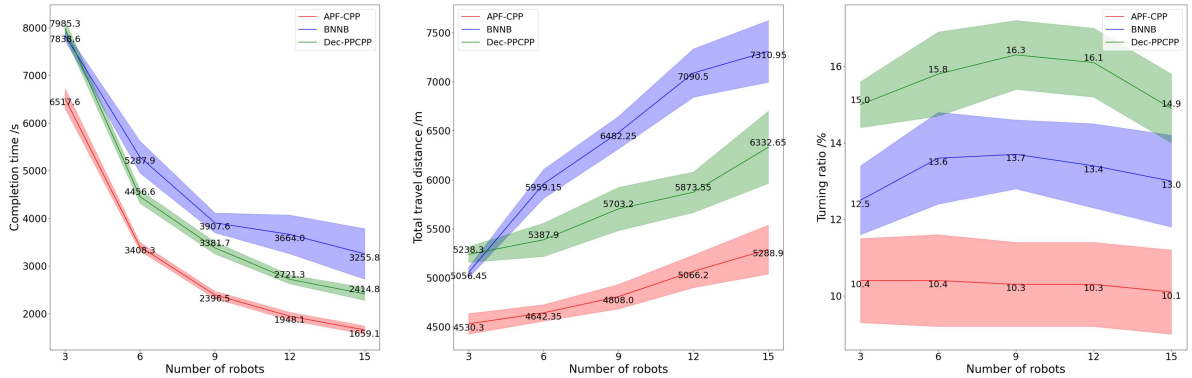
Fig. 5 illustrates these experiments' completion time, total travel distance, and turning ratio. The overall performance of APF-CPP is superior to the other two baselines. It can be observed that as the number of robots increases, the advantage of APF-CPP over BNNB and Dec-PPCPP becomes increasingly prominent. The advantage of APF-CPP over BNNB in completion time rises from 10.41% to 27.66%; over Dec-PPCPP, it increases from 13.51% to 16.48%. Similarly, the advantage in total coverage path length increases from 16.85% to 49.03% over BNNB and from 18.38% to 31.30% over Dec-PPCPP when the number of robots increases from 3 to 15. These results illustrate that the proposed method possesses a superior ability to coordinate between robots, enabling better utilization of each robot's capabilities within the multi-robot system. Furthermore, the proposed method outperforms the two baselines regarding the turning ratio metric. This indicates that the proposed method can significantly reduce the path turns and generate more efficient and regular coverage paths.

Apart from the three metrics mentioned above, we measured the average time, peak time for one single calculation, and the average total computation time for the entire task of the proposed method. The results shown in Table I demonstrate that our method exhibits effective real-time performance.

2) Case Study 2: Coverage Under Different Working Areas: In this study, two groups of experiments were conducted under large regions with different environment setups to verify the scalability of the APF-CPP algorithm. In each group, 30 robots participated. The initial positions of the robots were randomly distributed around the border of the maps, and the robots' parameters were set the same as in Case Study 1. For each group, 20 tests were carried out. Fig. 6 illustrates the performance of APF-CPP and the other two baselines in different working areas, and it is evident that the APF-CPP remains better than the two baselines.

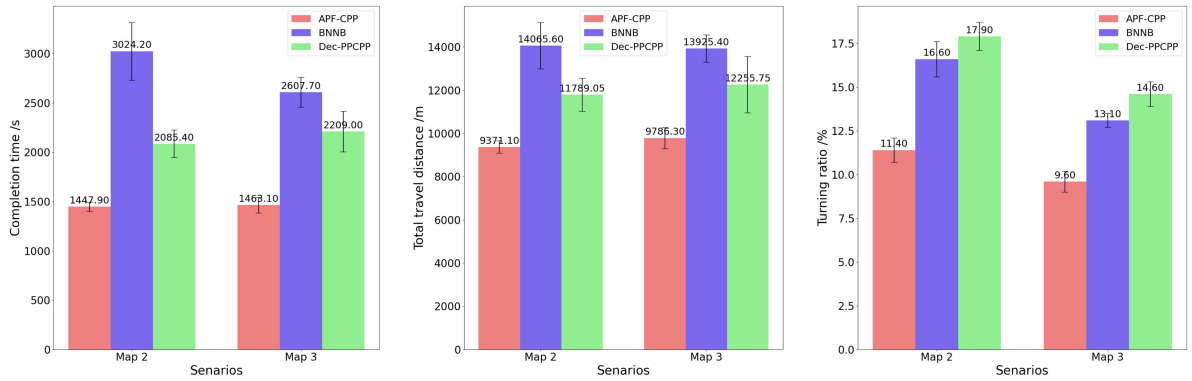
We conducted computation time analysis like Case Study 1. The results shown in Table II demonstrate that even when the number of participating robots and accessible cells is doubled, the proposed algorithm maintains its time efficiency, indicating its scalability. Furthermore, although the number of accessible

Authorized licensed use limited to: International Institute of Information Technology Bangalore. Downloaded on April 14,2026 at 17:42:39 UTC from IEEE Xplore. Restrictions apply.



(a) (b) (c)

Fig. 5. The results of the experiments with varying numbers of robots. Fig. 5(a) illustrates the completion time, Fig. 5(b) shows the total length of the coverage paths, and Fig. 5(c) displays the average turning ratio of each robot. The mean and standard deviation are indicated.



(a) (b) (c)

Fig. 6. The results of the experiments with different working areas. Fig. 6(a) illustrates the completion time, Fig. 6(b) shows the total length of the coverage paths, and Fig. 6(c) displays the average turning ratio of each robot. The mean and standard deviation are indicated.

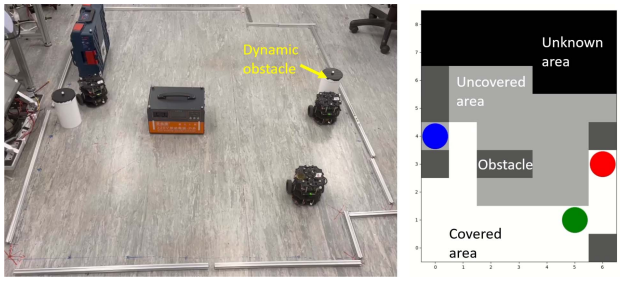


Fig. 7. Snapshot of the real-world experiment of APF-CPP (full video available in the multimedia material).

cells in the two target coverage environments is nearly the same, the computation burden for map 3 is heavier. This is due to the narrower environment setup, which increases the difficulty of calculating the point-to-point path planner.

B. Tests on Real Multi-Robot System Platform

Real-world experiments were conducted in a laboratory setting to validate the APF-CPP method's practicality and ability to handle dynamic obstacles in real-world scenarios. The target coverage area was partitioned into a 7×9 grid structure, and d was set as 0.25 m. Three Turtlebot3 Burger mobile robots were used to test the feasibility of the APF-CPP algorithm. The Raspberry Pi 4 board was installed on each Turtlebot3 for information sharing, and one of the Turtlebot3 was chosen as the central server to run the planner. The communication between robots and the central server was achieved via the Robot Operating System (ROS). The Optitrack Motion Capture system provided information on the positioning of the robots and dynamic obstacles. We only send the cells' occupation information inside the robots' detection range (set as 0.5 m in the experiments) to the robots to mimic the map-building process. The safety range was defined as 0.15 m. The linear velocity of the three robots was set as 0.12 m/s, 0.08 m/s, and 0.04 m/s; the turning speed of the three robots is set as 1 rd/s, and d_n is set as d. Fig. 7 demonstrates the real-world implementation of the proposed method. During the coverage process, dynamic obstacles were added to test the effectiveness of the uncertainty-handling mechanism. The result shows that the proposed method is robust in the face of uncertainties in real-world implementations.

Authorized licensed use limited to: International Institute of Information Technology Bangalore. Downloaded on April 14,2026 at 17:42:39 UTC from IEEE Xplore. Restrictions apply.

V. Conclusion

The APF-CPP approach has been demonstrated to be effective for a multi-robot system engaged in online coverage tasks. By utilizing attractive and repulsive forces, the robots tend to select a coverage policy or backtrack goal that promotes dispersion among robots, resulting in improved task allocation. During the execution of coverage tasks, each robot adheres to an overall movement direction, which reduces the number of turns in the paths. The updates to the coverage policy occur only at the initiation of the coverage task or when a robot encounters a dead-end situation. This coverage policy update mechanism enables the multi-robot system to adapt more effectively to unknown environments and prevents the inefficiency caused by frequent changes in coverage policy.

The APF-CPP approach demonstrated its effectiveness and advantages compared to SOTA methods through a diverse range of case studies. These studies encompassed various scenarios, including variations in operational velocity, different numbers of robots, and working areas of varying sizes. The simulation results provided compelling evidence of the benefits offered by the proposed method, including reduced completion time, minimized total coverage path distance, and improved turning ratio. Moreover, the method's real-time performance and scalability were successfully validated through these simulations. In addition, real-world experiments further emphasized the system's robustness in the face of uncertainties commonly encountered in practical environments. These experiments demonstrated the practicality and viability of the proposed method in real-world scenarios.

Future research will prioritize improving time efficiency in APF-CPP by optimizing the dead-end recovery mechanism. This enhancement is particularly vital as larger environments impose greater computational demands. Additionally, we intend to investigate task allocation mechanisms before initiating the coverage task to mitigate trajectory overlaps and minimize the risk of robot collisions. Moreover, we also plan to expand the capabilities of APF-CPP to accommodate distributed systems and address scenarios involving persistent coverage [27] and target coverage [28]. These advancements will empower APF-CPP to effectively tackle intricate coverage tasks with a higher degree of comprehensiveness.

References

[1] N. Karapetyan, K. Benson, C. McKinney, P. Taslakian, and I. Rekleitis, "Efficient multi-robot coverage of a known environment," in *Proc. 2017 IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2017, pp. 1846–1852.

[2] D.-H. Lee, "Resource-based task allocation for multi-robot systems," *Robot. Auton. Syst.*, vol. 103, pp. 151–161, 2018.

[3] L. Collins, P. Ghassemi, E. T. Esfahani, D. Doermann, K. Dantu, and S. Chowdhury, "Scalable coverage path planning of multi-robot teams for monitoring non-convex areas," in *Proc. IEEE 2021 Int. Conf. Robot. Automat.*, 2021, pp. 7393–7399.

[4] P. Maini, B. M. Gonultas, and V. Isler, "Online coverage planning for an autonomous weed mowing robot with curvature constraints," *IEEE Robot. Automat. Lett.*, vol. 7, no. 2, pp. 5445–5452, Apr. 2022.

[5] P. Ghassemi, D. DePauw, and S. Chowdhury, "Decentralized dynamic task allocation in swarm robotic systems for disaster response," in *Proc. 2019 IEEE Int. Symp. Multi-robot Multi-agent Syst.*, 2019, pp. 83–85.

[6] P. Yao, Q. Zhu, and R. Zhao, "Gaussian mixture model and self-organizing map neural-network-based coverage for target search in curve-shape area," *IEEE Trans. Cybern.*, vol. 52, no. 5, pp. 3971–3983, May 2022.

[7] W. Burgard, M. Moors, C. Stachniss, and F. E. Schneider, "Coordinated multi-robot exploration," *IEEE Trans. Robot.*, vol. 21, no. 3, pp. 376–386, Jun. 2005.

[8] L. Jiao, Z. Peng, L. Xi, S. Ding, and J. Cui, "Multi-agent coverage path planning via proximity interaction and cooperation," *IEEE Sensors J.*, vol. 22, no. 6, pp. 6196–6207, Mar. 2022.

[9] N. Hazon and G. A. Kaminka, "Redundancy, efficiency and robustness in multi-robot coverage," in *Proc. 2005 IEEE Int. Conf. Robot. Automat.*, 2005, pp. 735–741.

[10] J. Tang, C. Sun, and X. Zhang, "MSTC*: Multi-robot coverage path planning under physical constrain," in *Proc. 2021 IEEE Int. Conf. Robot. Automat.*, 2021, pp. 2518–2524.

[11] J. Lu, B. Zeng, J. Tang, T. L. Lam, and J. Wen, "TMSTC*: A path planning algorithm for minimizing turns in multi-robot coverage," *IEEE Robot. Automat. Lett.*, vol. 8, no. 8, pp. 5275–5282, Aug. 2023.

[12] I. Vandermeulen, R. Groß, and A. Kolling, "Turn-minimizing multirobot coverage," in *Proc. 2019 IEEE Int. Conf. Robot. Automat.*, 2019, pp. 1014–1020.

[13] Y. Tang, R. Zhou, G. Sun, B. Di, and R. Xiong, "A novel cooperative path planning for multi-robot persistent coverage in complex environments," *IEEE Sensors J.*, vol. 20, no. 8, pp. 4485–4495, Apr. 2020.

[14] T. Kusnur and M. Likhachev, "Complete, decomposition-free coverage path planning," in *2022 IEEE 18th Int. Conf. Automat. Sci. Eng.*, 2022, pp. 1431–1437.

[15] H. Song et al., "Multi-UAV disaster environment coverage planning with limited-endurance," in *Proc. 2022 IEEE Int. Conf. Robot. Automat.*, 2022, pp. 10760–10766.

[16] E. Galceran, R. Campos, N. Palomeras, M. Carreras, and P. Ridao, "Coverage path planning with realtime replanning for inspection of 3D underwater structures," in *Proc. 2014 IEEE Int. Conf. Robot. Automat.*, 2014, pp. 6586–6591.

[17] A. Ianenko, A. Artamonov, G. Sarapulov, A. Safaraleev, S. Bogomolov, and D.-K. Noh, "Coverage path planning with proximal policy optimization in a grid-based environment," in *Proc. 2020 59th IEEE Conf. Decis. Control*, 2020, pp. 4099–4104.

[18] H. Liu, J. Ma, and W. Huang, "Sensor-based complete coverage path planning in a dynamic environment for cleaning robot," *CAAI Trans. Intell. Technol.*, vol. 3, no. 1, pp. 65–72, 2018.

[19] R. Mitra and I. Saha, "Scalable online coverage path planning for multi-robot systems," in *Proc. 2022 IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2022, pp. 10102–10109.

[20] J. Hu, H. Niu, J. Carrasco, B. Lennox, and F. Arvin, "Voronoi-based multi-robot autonomous exploration in unknown environments via deep reinforcement learning," *IEEE Trans. Veh. Technol.*, vol. 69, no. 12, pp. 14413–14423, Dec. 2020.

[21] M. Hassan, D. Mustafic, and D. Liu, "Dec-PPCPP: A decentralized predator–prey-based approach to adaptive coverage path planning amid moving obstacles," in *Proc. 2020 IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2020, pp. 11732–11739.

[22] M. Hassan and D. Liu, "PPCPP: A predator–prey-based approach to adaptive coverage path planning," *IEEE Trans. Robot.*, vol. 36, no. 1, pp. 284–301, Feb. 2020.

[23] J. Song, S. Gupta, and J. Hare, "Game-theoretic cooperative coverage using autonomous vehicles," in *2014 Oceans-St John's*, 2014, pp. 1–6.

[24] J. Song and S. Gupta, "CARE: Cooperative autonomy for resilience and efficiency of robot teams for complete coverage of unknown environments under robot failures," *Auton. Robots*, vol. 44, pp. 647–671, 2020.

[25] J. Song and S. Gupta, "ϵ*: An online coverage path planning algorithm," *IEEE Trans. Robot.*, vol. 34, no. 2, pp. 526–533, Apr. 2018.

[26] D. Harabor and A. Grastien, "Online graph pruning for pathfinding on grid maps," in *Proc. AAAI Conf. Artif. Intell.*, 2011, pp. 1114–1119.

[27] Y. Tang, R. Zhou, G. Sun, B. Di, and R. Xiong, "A novel cooperative path planning for multirobot persistent coverage in complex environments," *IEEE Sensors J.*, vol. 20, no. 8, pp. 4485–4495, Apr. 2020.

[28] J. Li, Y. Xiong, and J. She, "UAV path planning for target coverage task in dynamic environment," *IEEE Internet Things J.*, vol. 10, no. 20, pp. 17734–17745, Oct. 2023.

Authorized licensed use limited to: International Institute of Information Technology Bangalore. Downloaded on April 14,2026 at 17:42:39 UTC from IEEE Xplore. Restrictions apply.